```
[1]: !pip install gensim

Requirement already satisfied: gensim in /home/jmittap/anaconda3/lib/python3.8/site-packages (4.3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /home/jmittap/anaconda3/lib/python3.8/site-packages (from gen
sim) (1.24.3)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /home/jmittap/anaconda3/lib/python3.8/site-packages (from g
ensim) (1.9.1)
Requirement already satisfied: smart-open>=1.8.1 in /home/jmittap/anaconda3/lib/python3.8/site-packages (from gens
im) (5.2.1)

[notice] A new release of pip is available: 25.0 -> 25.0.1
[notice] To update, run: pip install --upgrade pip
```

Figure 1: Gensim installation

```
•[19]: corpus = [
           "Jishnu is amazing", "Jishnu loves machine learning",
           "Deep learning is beautiful",
           "Word2Vec is a great way of creating word embeddings",
           "Gensim makes NLP easy", "Deep learning is a part of machine learning."
       ]

       tokenized_corpus = [simple_preprocess(sentence) for sentence in corpus]
       print(tokenized_corpus)

       [['jishnu', 'is', 'amazing'], ['jishnu', 'loves', 'machine', 'learning'], ['deep', 'learning', 'is', 'beautiful'],
       ['word', 'vec', 'is', 'great', 'way', 'of', 'creating', 'word', 'embeddings'], ['gensim', 'makes', 'nlp', 'easy'],
       ['deep', 'learning', 'is', 'part', 'of', 'machine', 'learning']]
```

Figure 2: Corpus creation and tokenisation

```
!pip install gensim
```

We now need to create a corpus. I have created a corpus of some random sentences as shown below:

```
corpus = [
    "Jishnu is amazing", "Jishnu loves machine learning",
    "Deep learning is beautiful",
    "Word2Vec is a great way of creating word embeddings",
    "Gensim makes NLP easy", "Deep learning is a part of machine learning."
]
```

The next step is the tokenisation process. We need to tokenise the corpus that we have. Here we are doing word level tokenisation, i.e., we are splitting the corpus into words. We can do this using the following code as shown in Figure 2.

```
from gensim.utils import simple_preprocess
tokenized_corpus = [simple_preprocess(sentence) for sentence in corpus]
print(tokenized_corpus)
```

Now you need to train your Word2Vec model. This can be done using the Gensim library that we installed and the functions in it, as shown in the code below.

```
from gensim.models import Word2Vec
model = Word2Vec(sentences=tokenized_corpus, vector_size=20, window=4, min_count=1, workers=4)
```

As we discussed, any word can be represented by a vector in our model. We can do that using the following lines of code as shown in Figure 3.

```
[24]: vector = model.wv['learning']
      print(vector)

[ 0.03655883  0.02535131  0.03378846  0.00381433  0.03175445 -0.01702683
 -0.00473201  0.02884287 -0.03760819 -0.01968052 -0.03755791 -0.00465021
  0.04769059 -0.03659583 -0.01166884 -0.00968871  0.04038718 -0.02965448
  0.00022581 -0.02376867]
```

Figure 3: 'Learning' represented as a vector

```
vector = model.wv['learning']
print(vector)
```

The figure shows how we can represent the word 'learning'.

We can also get the words that are closest to a given word in the trained vector space. Let us get the top three words similar to the word 'machine'. We can do this using the following code, as shown in Figure 4.

```
similar_words = model.wv.most_similar('machine', topn=3)
print(similar_words)
```

This is it! You can create your Word2Vec model now!